ОТЧЁТ

по лабораторной работе №2

«Коллекции, строки и функции в Python»

Вариант 20. Заказы доставки

1. Цель работы

Получить практические навыки работы с основными структурами данных языка Python и научиться применять их для обработки наборов информации. В процессе выполнения лабораторной работы необходимо освоить списки (list), кортежи (tuple), множества (set), словари (dict), операции над коллекциями, индексацию и срезы, циклическую обработку коллекций, включения списков (list comprehensions), обработку строк, функции для обработки коллекций, сортировку и поиск данных.

2. Постановка индивидуального задания

Вариант 20. Заказы доставки. Разработать консольное приложение для обработки информации о заказах доставки. Для каждого заказа хранить: номер; город; расстояние; стоимость; статус доставки.

Программа должна:

– найти заказы по городу;

– выполнить фильтрацию по расстоянию;

– определить среднюю стоимость;

– определить общий километраж;

– вывести список уникальных городов доставки;

– отсортировать заказы по расстоянию.

Согласно общим требованиям методических указаний программа получает от пользователя набор исходных данных, сохраняет объекты в

коллекциях, выполняет поиск, фильтрацию и сортировку, вычисляет статистические характеристики, определяет уникальные значения и выводит результаты. В работе использованы list, dict, set, кортеж допустимых статусов, пользовательские функции и циклы. Базы данных, Pandas, NumPy и сторонние библиотеки не применяются.

Для выполнения дополнительного задания реализованы list comprehension минимум в двух операциях, удаление и изменение объекта, фильтрация по двум критериям, сортировка по двум параметрам и группировка заказов по городу.

3. Краткие теоретические сведения

Коллекция — это структура, которая хранит набор значений. В реальных программах неудобно создавать отдельную переменную для каждого объекта, поэтому данные объединяют в список, кортеж, множество или словарь.

Список (list) — упорядоченная изменяемая последовательность. Элементы доступны по индексу, начиная с нуля; отрицательный индекс отсчитывается с конца. Срез sequence[start:stop:step] возвращает часть последовательности. Добавление выполняется методами append() и extend(), удаление — remove() и pop(). Функция len() возвращает количество элементов, оператор in проверяет наличие значения.

Кортеж (tuple) похож на список, но после создания его нельзя изменять. Кортежи удобны для хранения связанных значений, которые должны оставаться постоянными, например допустимых статусов доставки.

Множество (set) содержит только уникальные элементы. Повторяющиеся значения автоматически отбрасываются, поэтому set удобно использовать для списка городов доставки. Над множествами выполняются объединение (|), пересечение (&), разность (-) и симметрическая разность (^).

Словарь (dict) хранит пары «ключ → значение». Ключ должен быть уникальным: два одинаковых ключа в одном словаре быть не могут. Значение получают как student["name"] или через get(), который позволяет задать значение по умолчанию. Перебор пар выполняется методом items().

Список словарей — удобная модель набора однотипных объектов. Каждый заказ в программе представлен словарём с полями number, city, distance, cost и status, а все заказы хранятся в списке. Такая структура далее встречается при работе с JSON и REST API.

List comprehension позволяет компактно создать новый список: [order for order in orders if order["city"] == city]. Его стоит применять, когда выражение короткое и понятное; сложную логику лучше оставлять в обычном цикле. Функция sorted() возвращает новый отсортированный список, а метод sort() изменяет исходный. Функция enumerate() одновременно даёт индекс и элемент при переборе.

4. Описание программы

Консольное приложение zakazy_dostavki.py обрабатывает набор заказов доставки. Основная коллекция — список словарей orders. Допустимые статусы заданы кортежем STATUSES = ("оформлен", "в пути", "доставлен", "отменён"). Уникальные города получаются преобразованием списка городов во множество set.

Программа построена по модели методички: ввод данных, формирование коллекции, проверка, поиск, фильтрация, сортировка, статистический анализ и вывод. Меню реализовано циклом while и конструкцией if / elif / else. Пока пользователь не выберет пункт 0, можно многократно выполнять любые операции без перезапуска.

Основные функции получают данные через параметры и не опираются на глобальные переменные. find_by_city() и filter_by_distance() используют list comprehension. calculate_average_cost() и calculate_total_distance() считают

сумму циклом, как требуется методичкой, хотя для тех же задач допустимы встроенные sum() и len(). sort_by_distance() вызывает sorted() и не портит исходный порядок. unique_cities() возвращает set, group_by_city() формирует словарь списков.

Проверка ввода выполняется без аварийного завершения: номер и город не могут быть пустыми, номер заказа должен быть уникальным, расстояние — положительным числом, стоимость — неотрицательной (допускается бесплатная доставка). Дробные числа можно вводить через точку или запятую. Некорректный пункт меню и операции над пустой коллекцией сопровождаются сообщениями об ошибке.

Таблица 1 – Исходные данные тестового набора

Номер	Город	Расстояние, км	Стоимость, руб.	Статус
D-1001	Казань	12,50	450,00	оформлен
D-1002	Москва	850,00	3200,00	в пути
D-1003	Самара	45,00	700,00	доставлен
D-1004	Казань	8,00	350,00	доставлен
D-1005	Уфа	320,00	1800,00	оформлен
D-1006	Казань	12,50	480,00	в пути
D-1007	Тверь	0,10	0,00	доставлен

Набор содержит не менее пяти объектов (тест 1). У заказов D-1001, D-1004 и D-1006 совпадает город Казань, а у D-1001 и D-1006 ещё и расстояние 12,50 км (тест 2). Заказ D-1007 задаёт граничные значения: минимальное расстояние 0,10 км и нулевую стоимость (тест 3).

5. Исходный код программы

Листинг 1 – Файл zakazy_dostavki.py

```
"""
Лабораторная работа №2. Вариант 20.
Обработка заказов доставки.
"""


import sys


# Корректный вывод русских символов в консоли Windows
if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
```

```
    sys.stderr.reconfigure(encoding="utf-8")
    # При запуске из файла/конвейера читаем ввод как UTF-8
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")


# Кортеж допустимых статусов (значения не должны меняться в ходе работы)
STATUSES = ("оформлен", "в пути", "доставлен", "отменён")



def read_line(prompt):
    """Читает строку. В неинтерактивном режиме повторяет ввод в консоль."""
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



def read_positive_int(prompt):
    """Считывает целое положительное число и повторяет запрос при ошибке."""
    while True:
        text = read_line(prompt)
        if text.isdigit() and int(text) > 0:
            return int(text)
        print("Ошибка: введите целое положительное число.")



def is_non_negative_number(text):
    """Проверяет, что строка содержит неотрицательное вещественное число."""
    text = text.replace(",", ".", 1)
    if text.count(".") > 1:
        return False


    parts = text.split(".")
    if len(parts) == 1:
        return parts[0].isdigit()


    left = parts[0]
    right = parts[1]
    if left == "":
        left = "0"
    if right == "":
        return False
    return left.isdigit() and right.isdigit()



def read_non_negative_float(prompt):
    """Считывает неотрицательное вещественное число."""
    while True:
        text = read_line(prompt)
        if is_non_negative_number(text):
            return float(text.replace(",", ".", 1))
        print("Ошибка: введите число не меньше нуля (например, 12.5).")



def read_positive_float(prompt):
    """Считывает положительное вещественное число."""
    while True:
        value = read_non_negative_float(prompt)
        if value > 0:
            return value
        print("Ошибка: значение должно быть больше нуля.")



def read_non_empty(prompt):
```

```
    """Считывает непустую строку."""
    while True:
        text = read_line(prompt)
        if text:
            return text
        print("Ошибка: значение не должно быть пустым.")



def number_exists(orders, number):
    """Проверяет, есть ли уже заказ с таким номером."""
    for order in orders:
        if order["number"].lower() == number.lower():
            return True
    return False



def read_unique_number(orders, prompt):
    """Считывает уникальный номер заказа."""
    while True:
        number = read_non_empty(prompt)
        if number_exists(orders, number):
            print("Ошибка: заказ с таким номером уже существует.")
        else:
            return number



def read_status():
    """Считывает статус доставки из кортежа допустимых значений."""
    print("Статус доставки:")
    for index, status in enumerate(STATUSES, start=1):
        print(f"  {index}. {status}")


    while True:
        text = read_line("Выберите статус (1-4): ")
        if text.isdigit() and 1 <= int(text) <= len(STATUSES):
            return STATUSES[int(text) - 1]
        print("Ошибка: выберите номер статуса от 1 до 4.")



def input_order(orders):
    """Формирует словарь одного заказа по данным пользователя."""
    number = read_unique_number(orders, "  Номер заказа: ")
    city = read_non_empty("  Город доставки: ")
    distance = read_positive_float("  Расстояние, км: ")
    cost = read_non_negative_float("  Стоимость, руб.: ")
    status = read_status()
    return {
        "number": number,
        "city": city,
        "distance": distance,
        "cost": cost,
        "status": status,
    }



def print_orders(orders, title="Список заказов"):
    """Выводит коллекцию заказов в виде таблицы."""
    print(f"\n{title}:")
    if len(orders) == 0:
        print("Подходящих заказов нет.")
        return


    print(
        f"{'N':<4}{'Номер':<12}{'Город':<16}"
        f"{'Расст., км':<14}{'Стоимость':<14}{'Статус'}"
    )
    for index, order in enumerate(orders, start=1):
        print(
            f"{index:<4}{order['number']:<12}{order['city']:<16}"
            f"{order['distance']:<14.2f}{order['cost']:<14.2f}{order['status']}"
        )
```

```
def find_by_city(orders, city):
    """Ищет заказы по городу (list comprehension)."""
    city_lower = city.lower()
    return [
        order
        for order in orders
        if order["city"].lower() == city_lower
    ]



def find_by_number(orders, number):
    """Ищет заказ по номеру. Возвращает словарь или None."""
    number_lower = number.lower()
    for order in orders:
        if order["number"].lower() == number_lower:
            return order
    return None



def filter_by_distance(orders, limit, mode):
    """Фильтрует заказы по расстоянию (list comprehension)."""
    if mode == "max":
        return [
            order
            for order in orders
            if order["distance"] <= limit
        ]
    return [
        order
        for order in orders
        if order["distance"] >= limit
    ]



def filter_by_distance_and_status(orders, limit, status):
    """Дополнительная фильтрация сразу по двум критериям."""
    return [
        order
        for order in orders
        if order["distance"] <= limit and order["status"] == status
    ]



def sort_by_distance(orders, reverse=False):
    """Возвращает новый список, отсортированный по расстоянию."""
    return sorted(orders, key=lambda order: order["distance"], reverse=reverse)



def sort_by_distance_and_cost(orders):
    """Сортировка по двум параметрам: расстояние, затем стоимость."""
    return sorted(orders, key=lambda order: (order["distance"], order["cost"]))



def calculate_average_cost(orders):
    """Средняя стоимость. Считается циклом, как требуется методичкой."""
    total = 0
    for order in orders:
        total += order["cost"]
    return total / len(orders)



def calculate_total_distance(orders):
    """Общий километраж всех заказов."""
    total = 0
    for order in orders:
```

```
        total += order["distance"]
    return total


def find_extreme_orders(orders, field):
    """Находит заказы с минимальным и максимальным значением поля."""
    min_order = orders[0]
    max_order = orders[0]
    for order in orders:
        if order[field] < min_order[field]:
            min_order = order
        if order[field] > max_order[field]:
            max_order = order
    return min_order, max_order


def unique_cities(orders):
    """Возвращает множество уникальных городов доставки."""
    cities = [order["city"] for order in orders]
    return set(cities)


def unique_statuses(orders):
    """Возвращает множество уникальных статусов."""
    return {order["status"] for order in orders}


def group_by_city(orders):
    """Группирует заказы по городу в словарь списков."""
    groups = {}
    for order in orders:
        city = order["city"]
        if city not in groups:
            groups[city] = []
        groups[city].append(order)
    return groups


def add_orders(orders):
    """Добавляет один или несколько заказов в список."""
    count = read_positive_int("Сколько заказов добавить: ")
    for i in range(count):
        print(f"\nЗаказ {i + 1} из {count}:")
        order = input_order(orders)
        orders.append(order)
        print("Заказ добавлен.")


def search_orders(orders):
    """Поиск заказов по городу или по номеру."""
    print("\nПоиск:")
    print("1. По городу")
    print("2. По номеру заказа")
    choice = read_line("Выберите вид поиска: ")

    if choice == "1":
        city = read_non_empty("Введите город: ")
        found = find_by_city(orders, city)
        print_orders(found, f"Заказы в городе {city}")
    elif choice == "2":
        number = read_non_empty("Введите номер заказа: ")
        found = find_by_number(orders, number)
        if found is None:
            print("Заказ с таким номером не найден.")
        else:
            print_orders([found], f"Заказ {number}")
    else:
        print("Ошибка: выберите 1 или 2.")
```

```
def filter_orders(orders):
    """Фильтрация по расстоянию или по двум критериям."""
    print("\nФильтрация:")
    print("1. По расстоянию")
    print("2. По расстоянию и статусу")
    choice = read_line("Выберите вид фильтрации: ")


    if choice == "1":
        print("1. Расстояние не больше заданного")
        print("2. Расстояние не меньше заданного")
        mode_choice = read_line("Выберите условие: ")
        if mode_choice == "1":
            limit = read_non_negative_float("Введите максимальное расстояние, км: ")
            found = filter_by_distance(orders, limit, "max")
            print_orders(found, f"Заказы с расстоянием <= {limit:.2f} км")
        elif mode_choice == "2":
            limit = read_non_negative_float("Введите минимальное расстояние, км: ")
            found = filter_by_distance(orders, limit, "min")
            print_orders(found, f"Заказы с расстоянием >= {limit:.2f} км")
        else:
            print("Ошибка: выберите 1 или 2.")
    elif choice == "2":
        limit = read_non_negative_float("Введите максимальное расстояние, км: ")
        status = read_status()
        found = filter_by_distance_and_status(orders, limit, status)
        print_orders(
            found,
            f"Заказы со статусом «{status}» и расстоянием <= {limit:.2f} км",
        )
    else:
        print("Ошибка: выберите 1 или 2.")



def sort_orders(orders):
    """Сортировка коллекции без изменения исходного списка."""
    print("\nСортировка:")
    print("1. По расстоянию (по возрастанию)")
    print("2. По расстоянию (по убыванию)")
    print("3. По расстоянию и стоимости")
    choice = read_line("Выберите вид сортировки: ")


    if choice == "1":
        print_orders(sort_by_distance(orders), "Заказы по возрастанию расстояния")
    elif choice == "2":
        print_orders(
            sort_by_distance(orders, reverse=True),
            "Заказы по убыванию расстояния",
        )
    elif choice == "3":
        print_orders(
            sort_by_distance_and_cost(orders),
            "Заказы по расстоянию, затем по стоимости",
        )
    else:
        print("Ошибка: выберите пункт от 1 до 3.")



def print_statistics(orders):
    """Выводит статистические показатели по заказам."""
    count = len(orders)
    average_cost = calculate_average_cost(orders)
    total_distance = calculate_total_distance(orders)
    average_distance = total_distance / count
    cheapest, expensive = find_extreme_orders(orders, "cost")
    nearest, farthest = find_extreme_orders(orders, "distance")


    print("\n----- Статистика -----")
    print(f"Количество заказов: {count}")
    print(f"Средняя стоимость: {average_cost:.2f} руб.")
    print(f"Общий километраж: {total_distance:.2f} км")
    print(f"Среднее расстояние: {average_distance:.2f} км")
    print(
        f"Самый дешёвый заказ: {cheapest['number']} "
```

```
        f"({cheapest['cost']:.2f} руб., {cheapest['city']})"
    )
    print(
        f"Самый дорогой заказ: {expensive['number']} "
        f"({expensive['cost']:.2f} руб., {expensive['city']})"
    )
    print(
        f"Ближайший заказ: {nearest['number']} "
        f"({nearest['distance']:.2f} км)"
    )
    print(
        f"Самый дальний заказ: {farthest['number']} "
        f"({farthest['distance']:.2f} км)"
    )


    print("\nКоличество заказов по статусам:")
    for status in STATUSES:
        status_count = 0
        for order in orders:
            if order["status"] == status:
                status_count += 1
        print(f"  {status}: {status_count}")



def print_unique_values(orders):
    """Выводит уникальные города и статусы."""
    cities = unique_cities(orders)
    statuses = unique_statuses(orders)


    print("\n----- Уникальные значения -----")
    print(f"Города доставки ({len(cities)}):")
    for city in sorted(cities):
        print(f"  {city}")


    print(f"\nСтатусы в текущем наборе ({len(statuses)}):")
    for status in sorted(statuses):
        print(f"  {status}")



def edit_order(orders):
    """Изменяет выбранные поля существующего заказа."""
    number = read_non_empty("Введите номер заказа для изменения: ")
    order = find_by_number(orders, number)
    if order is None:
        print("Заказ с таким номером не найден.")
        return


    print_orders([order], "Текущие данные заказа")
    print("\nЧто изменить?")
    print("1. Город")
    print("2. Расстояние")
    print("3. Стоимость")
    print("4. Статус")
    choice = read_line("Выберите поле: ")


    if choice == "1":
        order["city"] = read_non_empty("Новый город: ")
    elif choice == "2":
        order["distance"] = read_positive_float("Новое расстояние, км: ")
    elif choice == "3":
        order["cost"] = read_non_negative_float("Новая стоимость, руб.: ")
    elif choice == "4":
        order["status"] = read_status()
    else:
        print("Ошибка: выберите пункт от 1 до 4.")
        return


    print("Заказ обновлён.")
    print_orders([order], "Обновлённые данные")
```

```
def delete_order(orders):
    """Удаляет заказ по номеру."""
    number = read_non_empty("Введите номер заказа для удаления: ")
    order = find_by_number(orders, number)
    if order is None:
        print("Заказ с таким номером не найден.")
        return


    print_orders([order], "Заказ будет удалён")
    confirm = read_line("Подтвердите удаление (да/нет): ")
    if confirm.lower() in ("да", "д", "yes", "y"):
        orders.remove(order)
        print("Заказ удалён.")
    else:
        print("Удаление отменено.")



def print_groups(orders):
    """Выводит группировку заказов по городам."""
    groups = group_by_city(orders)
    print("\n----- Группировка по городу -----")
    for city, city_orders in groups.items():
        city_distance = calculate_total_distance(city_orders)
        city_cost = 0
        for order in city_orders:
            city_cost += order["cost"]
        print(f"\n{city}: заказов {len(city_orders)}, "
              f"километраж {city_distance:.2f} км, "
              f"сумма {city_cost:.2f} руб.")
        print_orders(city_orders, f"Заказы города {city}")



def ensure_not_empty(orders):
    """Проверяет, что коллекция уже содержит данные."""
    if len(orders) == 0:
        print("Ошибка: сначала добавьте хотя бы один заказ.")
        return False
    return True



def print_menu():
    print()
    print("===== МЕНЮ =====")
    print("1. Добавить объект")
    print("2. Показать объекты")
    print("3. Найти объект")
    print("4. Фильтрация")
    print("5. Сортировка")
    print("6. Статистика")
    print("7. Уникальные значения")
    print("8. Изменить объект")
    print("9. Удалить объект")
    print("10. Группировка по городу")
    print("0. Выход")
    print()



def main():
    # Основная коллекция: список словарей
    orders = []


    while True:
        print_menu()
        choice = read_line("Выберите действие: ")


        if choice == "1":
            add_orders(orders)
        elif choice == "2":
            if ensure_not_empty(orders):
                print_orders(orders, "Все введённые заказы")
        elif choice == "3":
```

```
            if ensure_not_empty(orders):
                search_orders(orders)
        elif choice == "4":
            if ensure_not_empty(orders):
                filter_orders(orders)
        elif choice == "5":
            if ensure_not_empty(orders):
                sort_orders(orders)
        elif choice == "6":
            if ensure_not_empty(orders):
                print_statistics(orders)
        elif choice == "7":
            if ensure_not_empty(orders):
                print_unique_values(orders)
        elif choice == "8":
            if ensure_not_empty(orders):
                edit_order(orders)
        elif choice == "9":
            if ensure_not_empty(orders):
                delete_order(orders)
        elif choice == "10":
            if ensure_not_empty(orders):
                print_groups(orders)
        elif choice == "0":
            print("Работа программы завершена.")
            break
        else:
            print("Ошибка: выберите пункт меню от 0 до 10.")


if __name__ == "__main__":
    main()
```

6. Примеры работы программы

Ниже приведены снимки экрана консольного приложения и результаты тестирования, предусмотренные методическими указаниями.

```
C:\Users\Shmurdyakich\Desktop\python2>python zakazy_dostavki.py

===== МЕНЮ =====
1. Добавить объект
2. Показать объекты
3. Найти объект
4. Фильтрация
5. Сортировка
6. Статистика
7. Уникальные значения
8. Изменить объект
9. Удалить объект
10. Группировка по городу
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 1 – Главное меню программы

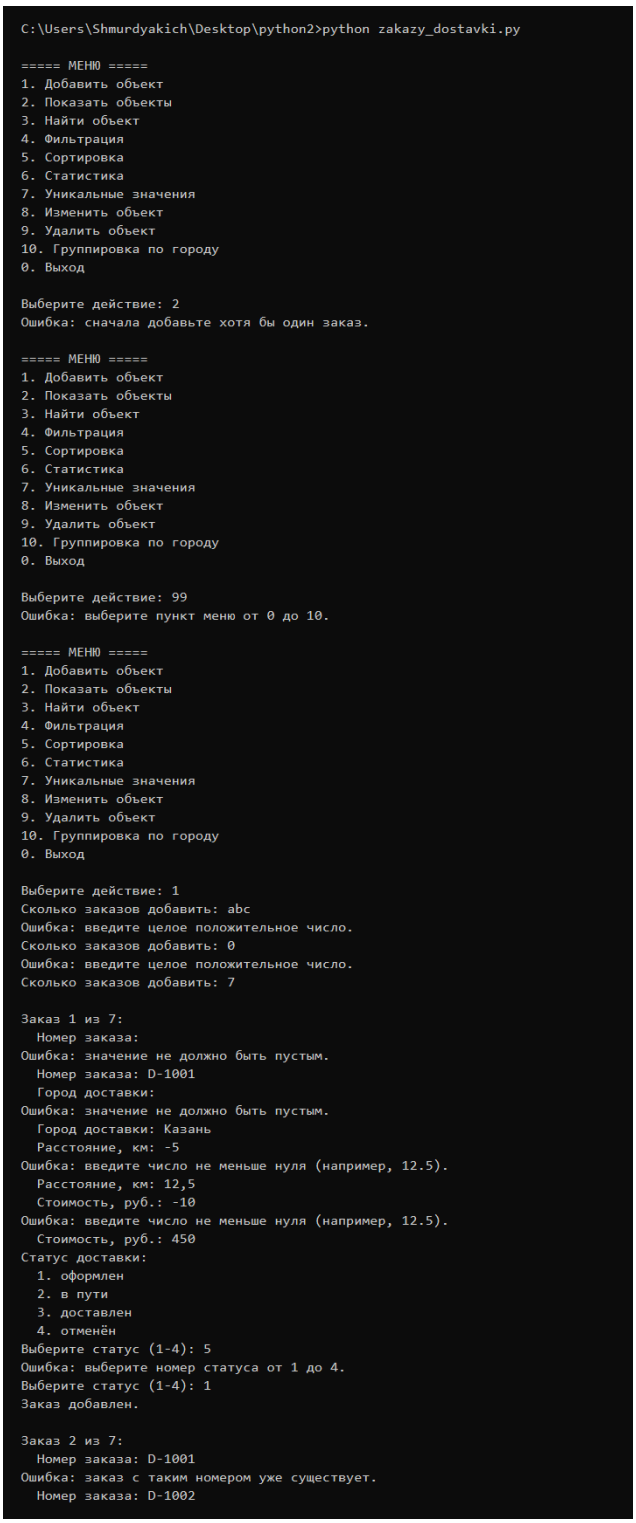


Рисунок 2 – Проверка корректности ввода

На рисунке 2 программа отклоняет просмотр пустой коллекции, несуществующий пункт меню, нечисловое и нулевое количество заказов, пустые номер и город, отрицательные расстояние и стоимость, недопустимый статус и повтор номера D-1001. После этого ввод принимается.

```
Выберите действие: 2

Все введённые заказы:
N   Номер       Город          Расст., км    Стоимость     Статус
1   D-1001      Казань         12.50         450.00        оформлен
2   D-1002      Москва         850.00        3200.00       в пути
3   D-1003      Самара         45.00         700.00        доставлен
4   D-1004      Казань         8.00          350.00        доставлен
5   D-1005      Уфа            320.00        1800.00       оформлен
6   D-1006      Казань         12.50         480.00        в пути
7   D-1007      Тверь          0.10          0.00          доставлен
```

Рисунок 3 – Тест 1. Вывод набора из семи заказов

Тест 1. В коллекцию добавлено семь заказов. В таблице видны все поля объекта: номер, город, расстояние, стоимость и статус.

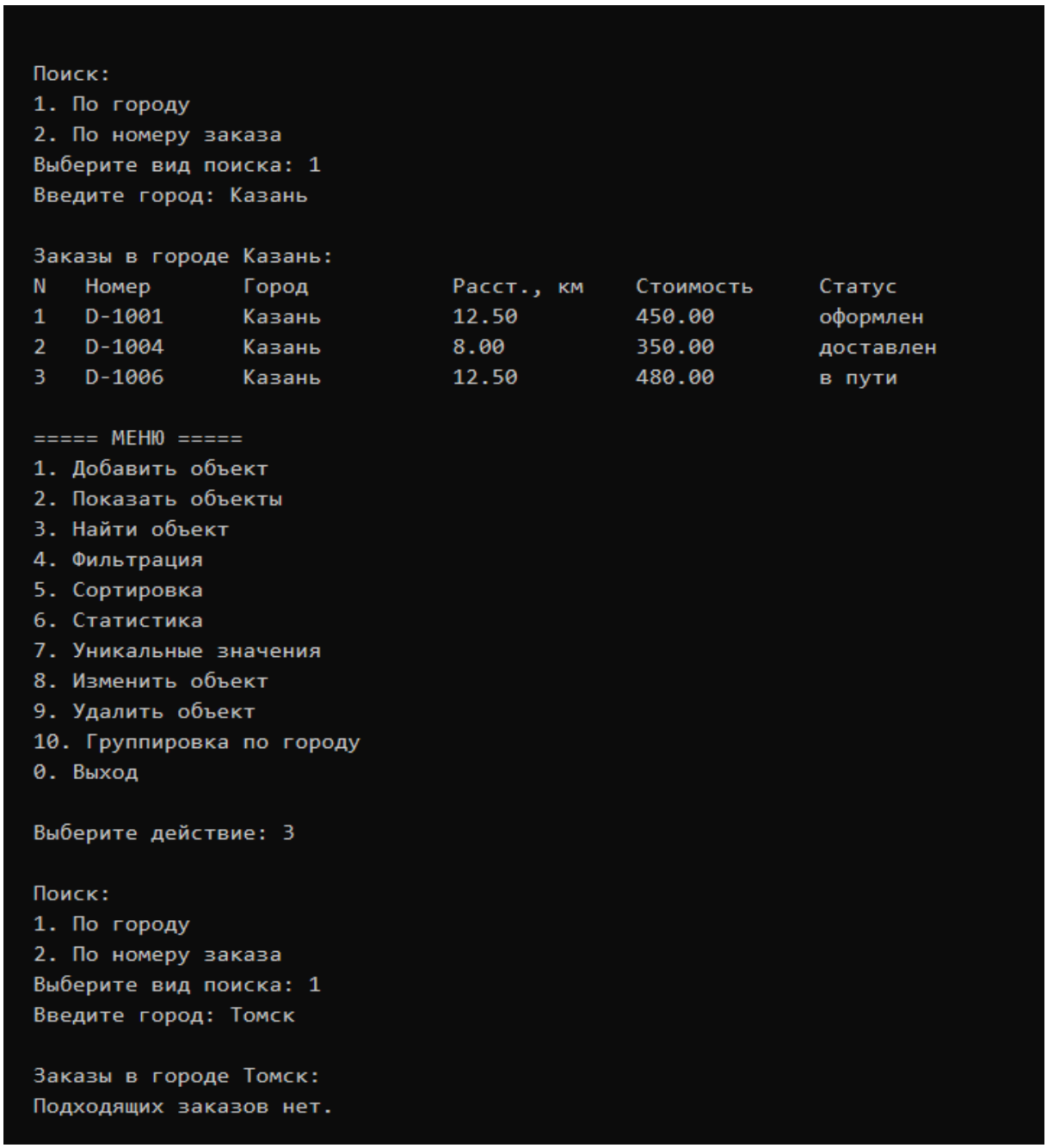


Рисунок 4 – Поиск по городу (тест 2) и пустой поиск (тест 4)

Тест 2. Поиск по городу «Казань» вернул три заказа, у двух из них расстояние одинаковое (12,50 км). Тест 4. Поиск по городу «Томск» не нашёл ни одного объекта: программа вывела сообщение «Подходящих заказов нет». Поиск по номеру НЕТ-999 также завершился без результата.

```
2. По расстоянию и статусу
Выберите вид фильтрации: 1
1. Расстояние не больше заданного
2. Расстояние не меньше заданного
Выберите условие: 1
Введите максимальное расстояние, км: 50

Заказы с расстоянием <= 50.00 км:
N   Номер       Город          Расст., км    Стоимость     Статус
1   D-1001      Казань         12.50         450.00        оформлен
2   D-1003      Самара         45.00         700.00        доставлен
3   D-1004      Казань         8.00          350.00        доставлен
4   D-1006      Казань         12.50         480.00        в пути
5   D-1007      Тверь          0.10          0.00          доставлен

===== МЕНЮ =====
1. Добавить объект
2. Показать объекты
3. Найти объект
4. Фильтрация
5. Сортировка
6. Статистика
7. Уникальные значения
8. Изменить объект
9. Удалить объект
10. Группировка по городу
0. Выход

Выберите действие: 4

Фильтрация:
1. По расстоянию
2. По расстоянию и статусу
Выберите вид фильтрации: 1
1. Расстояние не больше заданного
2. Расстояние не меньше заданного
Выберите условие: 1
Введите максимальное расстояние, км: 0.05

Заказы с расстоянием <= 0.05 км:
Подходящих заказов нет.

===== МЕНЮ =====
1. Добавить объект
```

Рисунок 5 – Фильтрация по расстоянию и пустой результат (тест 5)

Фильтрация «расстояние не больше 50 км» оставила пять ближних заказов. Тест 5. При пороге 0,05 км подходящих объектов нет, программа корректно сообщает об этом и не завершается с ошибкой.

```
2. По расстоянию (по убыванию)
3. По расстоянию и стоимости
Выберите вид сортировки: 1

Заказы по возрастанию расстояния:
N   Номер       Город           Расст., км    Стоимость     Статус
1   D-1007      Тверь           0.10          0.00          доставлен
2   D-1004      Казань          8.00          350.00        доставлен
3   D-1001      Казань          12.50         450.00        оформлен
4   D-1006      Казань          12.50         480.00        в пути
5   D-1003      Самара          45.00         700.00        доставлен
6   D-1005      Уфа             320.00        1800.00       оформлен
7   D-1002      Москва          850.00        3200.00       в пути

===== МЕНЮ =====
1. Добавить объект
```

Рисунок 6 – Сортировка заказов по расстоянию

Сортировка выполнена функцией sorted() по возрастанию расстояния. Первым оказался граничный заказ D-1007 (0,10 км), последним — D-1002 (850 км). Исходный список при этом не изменялся.

```
----- Статистика -----
Количество заказов: 7
Средняя стоимость: 997.14 руб.
Общий километраж: 1248.10 км
Среднее расстояние: 178.30 км
Самый дешёвый заказ: D-1007 (0.00 руб., Тверь)
Самый дорогой заказ: D-1002 (3200.00 руб., Москва)
Ближайший заказ: D-1007 (0.10 км)
Самый дальний заказ: D-1002 (850.00 км)

Количество заказов по статусам:
  оформлен: 2
  в пути: 2
  доставлен: 3
  отменён: 0

===== МЕНЮ =====
1. Добавить объект
2. Показать объекты
3. Найти объект
4. Фильтрация
5. Сортировка
6. Статистика
7. Уникальные значения
8. Изменить объект
9. Удалить объект
10. Группировка по городу
0. Выход

Выберите действие: 7

----- Уникальные значения -----
Города доставки (5):
  Казань
  Москва
  Самара
  Тверь
  Уфа

Статусы в текущем наборе (3):
  в пути
  доставлен
  оформлен
```

Рисунок 7 – Статистика и уникальные значения

Статистика: средняя стоимость 997,14 руб., общий километраж 1248,10 км, среднее расстояние 178,30 км. Самый дешёвый и ближайший заказ — D-1007, самый дорогой и дальний — D-1002. Уникальные города доставки: Казань, Москва, Самара, Тверь, Уфа.

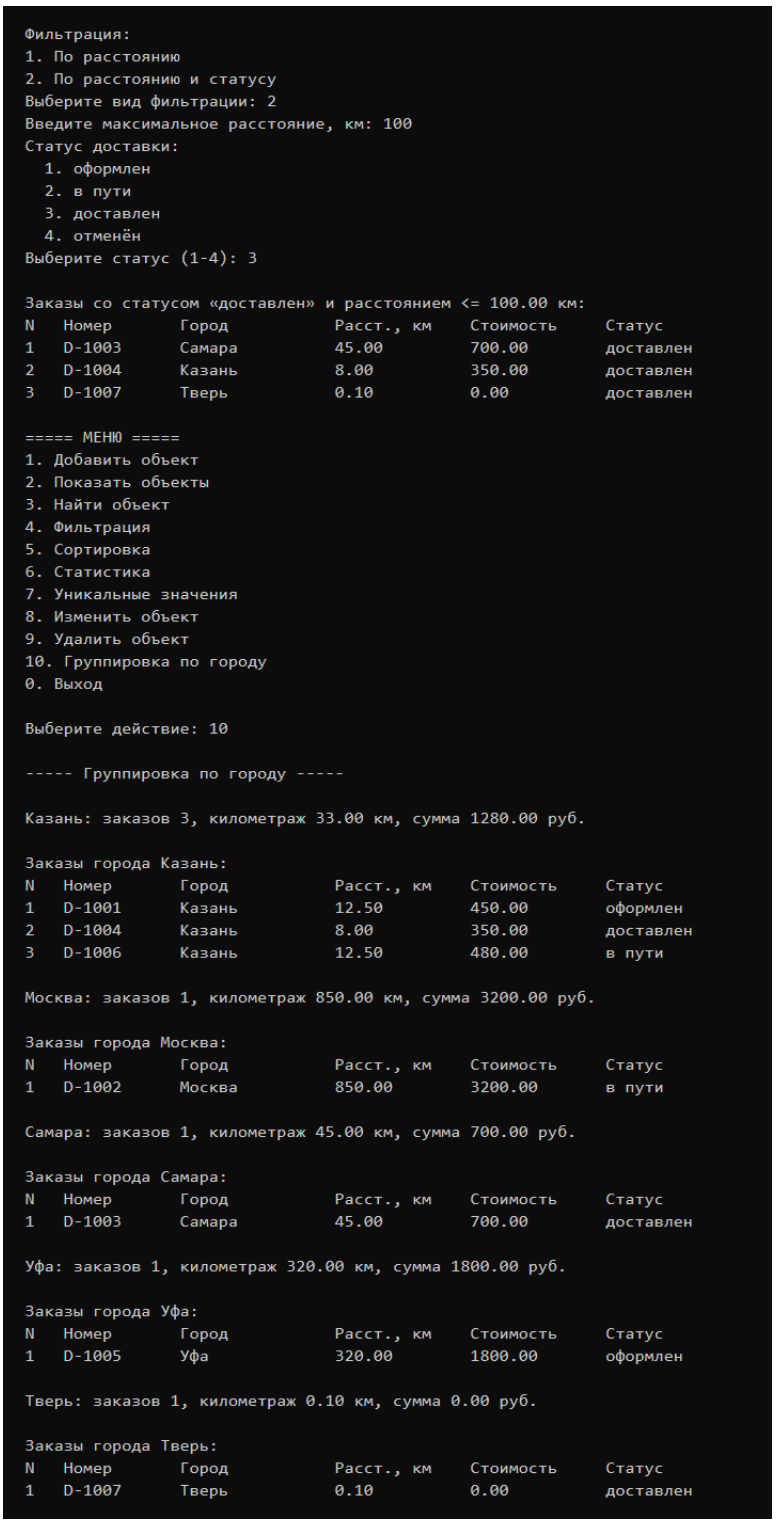


Рисунок 8 – Фильтрация по двум критериям и группировка

Дополнительно выполнены фильтрация сразу по расстоянию и статусу «доставлен» (заказы D-1003, D-1004, D-1007) и группировка по городу. Для Казани программа посчитала 3 заказа, 33,00 км и сумму 1280,00 руб.

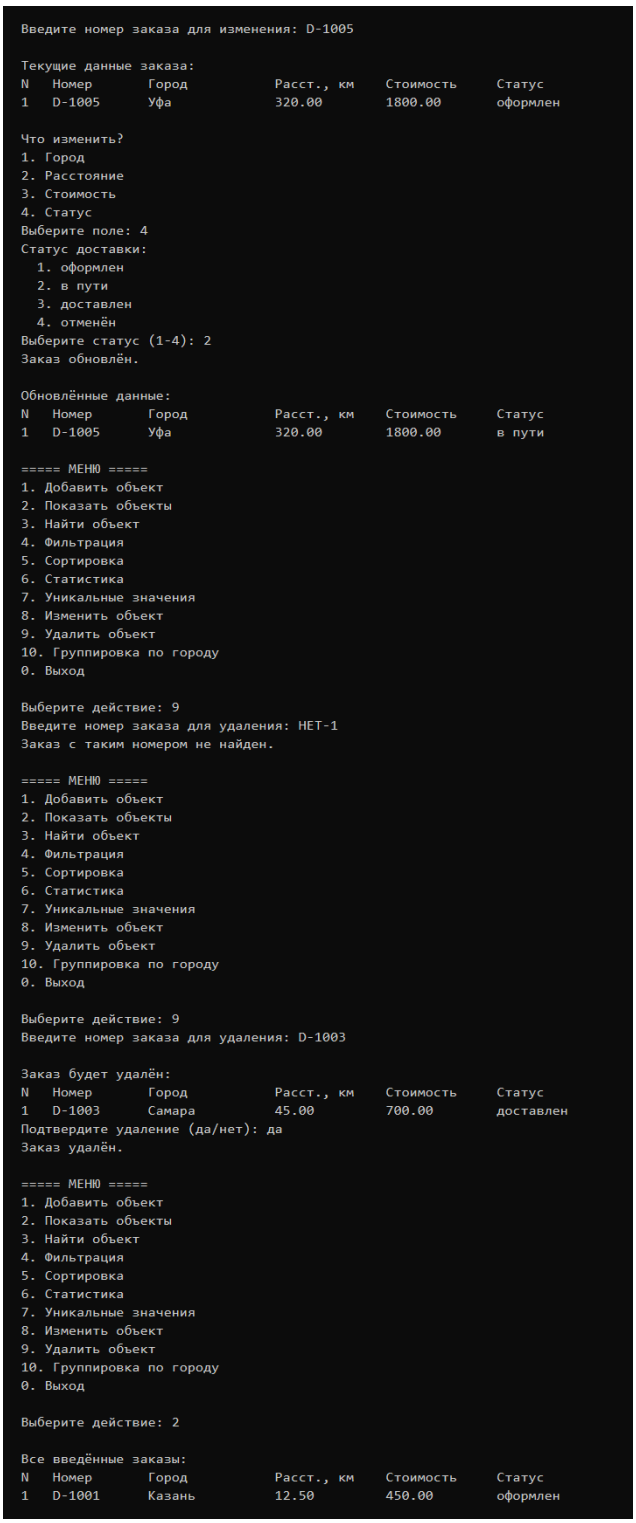


Рисунок 9 – Изменение и удаление заказа

Дополнительное задание: статус заказа D-1005 изменён с «оформлен» на «в пути», заказ D-1003 удалён после подтверждения. Повторный вывод списка показывает шесть оставшихся объектов, у D-1005 уже новый статус.

7. Ответы на контрольные вопросы

1. Что такое коллекция?

Коллекция — это структура данных, которая хранит набор элементов как единое целое. Вместо отдельных переменных student1, student2, student3 используют, например, список grades = [5, 4, 3, 5, 4]. Коллекции позволяют перебирать, искать, фильтровать и сортировать данные циклами и встроенными функциями.

2. Какие основные коллекции Python вы знаете?

Основные встроенные коллекции: list — упорядоченная изменяемая последовательность; tuple — упорядоченная неизменяемая последовательность; set — множество уникальных элементов; dict — пары «ключ → значение». В работе использованы все четыре типа.

3. Чем list отличается от tuple?

Список можно изменять после создания: заменять элементы, добавлять и удалять их. Кортеж после создания изменить нельзя. Оба типа упорядочены и поддерживают индексацию и срезы. Кортеж применяют, когда набор значений должен оставаться постоянным, в программе это кортеж статусов STATUSES.

4. Чем set отличается от list?

Множество хранит только уникальные элементы и не гарантирует порядок как список. В списке одно и то же значение может встречаться несколько раз, в множестве повтор будет отброшен. У множества нет индексов, зато есть операции объединения и пересечения. В работе set использован для списка уникальных городов.

5. Для чего используется dict?

Словарь хранит данные в виде пар ключ — значение и позволяет быстро получить значение по имени поля. Один заказ удобно представить словарём с ключами number, city, distance, cost и status вместо нескольких разрозненных переменных.

6. Что такое ключ словаря?

Ключ — это уникальное имя, по которому в словаре находят значение. Ключом может быть неизменяемый объект, чаще всего строка или число. Обращение student["name"] использует ключ "name".

7. Может ли словарь содержать два одинаковых ключа?

Нет. Ключи в одном словаре уникальны. Если присвоить значение уже существующему ключу, старое значение будет заменено новым. Поэтому у заказа не может быть двух полей city одновременно.

8. Как получить элемент списка по индексу?

Нужно указать индекс в квадратных скобках: numbers[0] вернёт первый элемент, numbers[2] — третий. Индексация начинается с нуля. Если индекса нет, возникнет IndexError.

9. Что такое отрицательная индексация?

Отрицательный индекс отсчитывает элементы с конца списка: numbers[-1] — последний элемент, numbers[-2] — предпоследний. Это удобно, когда нужно взять последний заказ без вычисления длины.

10. Что такое срез?

Срез — это часть последовательности, задаваемая синтаксисом sequence[start:stop:step]. Граница stop не включается. Пример: numbers[1:4] для [10, 20, 30, 40, 50] даёт [20, 30, 40]. Срез numbers[::2] берёт каждый второй элемент.

11. Для чего используется len()?

Функция len() возвращает количество элементов коллекции или символов строки. В программе она нужна, чтобы узнать число заказов, посчитать среднее (сумма / len(orders)) и проверить, пуста ли коллекция.

12. Как добавить элемент в список?

Один элемент добавляют методом append(): orders.append(order). Вставить элемент в заданную позицию можно методом insert(). Несколько элементов сразу добавляет extend().

13. Чем отличаются append() и extend()?

append() добавляет свой аргумент как один новый элемент, даже если это список. extend() разворачивает переданную последовательность и добавляет каждый её элемент отдельно. Поэтому numbers.append([50, 60]) даст вложенный список, а extend([50, 60]) — два обычных числа.

14. Как удалить элемент списка?

Метод remove(значение) удаляет первое вхождение указанного значения. Метод pop(индекс) удаляет элемент по индексу и возвращает его; pop() без аргумента удаляет последний элемент. Метод clear() очищает весь список. В программе заказ удаляется через orders.remove(order).

15. Чем отличаются remove() и pop()?

remove() ищет элемент по значению и ничего не возвращает. pop() работает по индексу и возвращает удалённый элемент, поэтому его можно сохранить в переменную. Если нужного значения нет, remove() вызывает ошибку; pop() с неверным индексом тоже вызывает ошибку.

16. Что делает оператор in?

Оператор in проверяет, содержится ли элемент в коллекции, и возвращает True или False. Пример: if 20 in numbers. Для словаря in проверяет наличие ключа: if "name" in student. Существует и противоположная проверка not in.

17. Как проверить наличие ключа в словаре?

Используется оператор in: if "name" in student. Такая проверка нужна перед обращением student["phone"], чтобы не получить KeyError, если ключа нет.

18. Для чего используется get()?

Метод get() возвращает значение по ключу и не вызывает ошибку, если ключа нет: тогда результатом будет None либо значение по умолчанию, заданное вторым аргументом. Пример: phone = student.get("phone", "Не указан").

19. Как перебрать словарь по ключам и значениям?

Удобнее всего цикл for key, value in student.items(). Можно перебирать только ключи (for key in student или student.keys()) либо только значения (student.values()).

20. Для чего используется items()?

Метод items() возвращает пары (ключ, значение). Вместе с распаковкой кортежа это даёт короткий цикл for key, value in order.items(), в котором доступны и имя поля, и его значение.

21. Что такое множество?

Множество (set) — неупорядоченная коллекция уникальных элементов. Повторы при создании отбрасываются: set([1, 2, 3, 3, 2]) даст {1, 2, 3}. Множества применяют, когда важно получить список различных значений, например городов доставки.

22. Как получить уникальные элементы последовательности?

Нужно преобразовать последовательность во множество: unique_cities = set(cities). В программе сначала собирается список городов list comprehension, затем он передаётся в set(). Повторяющаяся Казань в результате встречается один раз.

23. Что такое пересечение множеств?

Пересечение A & B — это элементы, которые входят и в A, и в B. Пример из методички: общие студенты двух групп group_a & group_b. Для

городов пересечение показало бы города, которые встречаются в обоих наборах заказов.

24. Что такое объединение множеств?

Объединение A | B содержит все элементы, которые есть хотя бы в одном из множеств. Повторы при этом не дублируются. Так можно сложить города двух выборок в один уникальный список.

25. Что такое list comprehension?

List comprehension — краткая запись создания списка: [выражение for элемент in последовательность if условие]. В программе так реализованы поиск по городу и фильтрация по расстоянию. Это тот же цикл for, записанный в одну конструкцию.

26. В каких случаях list comprehension улучшает читаемость программы?

Он улучшает читаемость, когда новый список получается из другого простым выражением и коротким условием. Длинные ветвления, несколько вложенных действий или сложные побочные эффекты лучше оставлять в обычном цикле: методичка прямо предупреждает не заменять comprehension любую обработку.

27. Как отсортировать список?

Функция sorted(items) возвращает новый отсортированный список. Метод items.sort() сортирует исходный список на месте. Можно задать ключ и направление: sorted(orders, key=lambda order: order["distance"], reverse=True).

28. Чем sort() отличается от sorted()?

sorted() не изменяет исходную коллекцию и возвращает новый список. sort() меняет тот же самый список и возвращает None. В программе используется sorted(), чтобы показ отсортированных заказов не ломал исходный порядок ввода.

29. Что делает enumerate()?

enumerate(sequence) при переборе одновременно даёт индекс и элемент. Цикл for index, order in enumerate(orders, start=1) нумерует заказы с единицы при выводе таблицы. Это предпочтительнее, чем for i in range(len(orders)).

30. Почему список словарей является удобной структурой для представления набора объектов?

Каждый объект описывается одним словарём с понятными именами полей, а набор объектов хранится в списке. Тогда поиск, фильтрация и сортировка записываются одинаково для любого количества заказов. Такую же модель используют JSON, REST API и таблицы баз данных: одна запись — один словарь, много записей — список словарей.

8. Вывод

В ходе лабораторной работы №2 разработано консольное приложение на языке Python для обработки заказов доставки (вариант 20). Программа хранит заказы как список словарей, использует кортеж статусов и множество уникальных городов, выполняет поиск по городу, фильтрацию по расстоянию, сортировку, расчёт средней стоимости и общего километража.

При реализации освоены коллекции list, tuple, set и dict, индексация, срезы, циклы, list comprehension, обработка строк и выделение операций в функции с параметрами. Меню построено циклом while по рекомендуемому в методичке интерфейсу. Дополнительно реализованы изменение и удаление заказа, фильтрация по двум критериям, сортировка по двум параметрам и группировка по городу. Программа проверена на обычном наборе, наборах с повторяющимися и граничными значениями, а также на пустом поиске и пустой фильтрации.